

Progetto reti logiche - Filtro Esponenziale

Alessandro Boglioli

1 Introduzione

Si progetti un circuito sequenziale per la realizzazione di un filtro esponenziale del secondo ordine. Il sistema riceve una sequenza continua di campioni X_t rappresentati in virgola fissa su 32 bit in cui 16 bit rappresentano la parte frazionaria e produce in uscita il segnale filtrato Y_t rappresentato con la stessa notazione e calcolato come:

$$Y_t = \alpha X_t + \alpha(1 - \alpha)Y_{t-1} + (1 - \alpha)^2 Y_{t-2}$$

Per semplicità si limita il coefficiente del filtro α alle potenze negative del 2, cioè:

$$\alpha = \frac{1}{2^k}$$

In questo modo il calcolo dei coefficienti α , $\alpha(1 - \alpha)$ e $(1 - \alpha)^2$ risulta notevolmente semplificato. Il valore k viene fornito in ingresso al sistema e si suppone costante durante l'elaborazione di una sequenza di ingressi. Data la precisione della rappresentazione di ingresso, si limita il valore di k all'intervallo $[0; 7]$. Il comportamento del filtro non è definito se il coefficiente k viene cambiato durante la normale operatività del filtro. Il sistema deve altresì disporre di un segnale di "inizializzazione" che riporta lo stato del filtro a zero.

Una volta realizzato il componente, è richiesto di realizzare un test-bench per la simulazione e la verifica del corretto funzionamento nei diversi casi. In particolare, è necessario verificare il comportamento del filtro rispetto alla risposta al gradino. La simulazione deve tenere conto dei ritardi reali del sistema (simulazione Post Place & Route).

Iniziamo elencando quali sono le ipotesi e assunzioni che ci poniamo per la risoluzione di questo problema:

- Si assume che il valore di α (quindi il valore di k) sia costante durante l'elaborazione di una sequenza
- Il comportamento del sistema non è definito se il valore di α viene modificato durante il funzionamento
- La sequenza di campioni in ingresso può variare in modo asincrono anche durante lo sviluppo di una sequenza di segnali filtrati in uscita
- Gli ingressi possono essere considerati sia positivi che negativi: 1 bit del segno, 15 per la parte intera e 16 per la parte decimale

Di seguito vengono riportate alcune limitazioni che andranno ad impattare sullo sviluppo dei risultati:

- La precisione è limitata dalla rappresentazione in virgola fissa a 32 bit, questo genererà delle imprecisioni nei risultati rispetto al mondo reale, in quanto dovremmo accontentarci di una precisione più bassa. Questa limitazione crea discrepanze anche tra soluzioni differenti per lo sviluppo dello stesso filtro esponenziale

2 Specifica

Iniziamo con lo sviluppo della formula con cui si calcola l'uscita Y_t , in modo da individuare un primo approccio da seguire per lo sviluppo del circuito logico.

$$\begin{aligned}Y_t &= \alpha X_t + \alpha(1 - \alpha)Y_{t-1} + (1 - \alpha)^2 Y_{t-2} \\Y_t &= \alpha X_t + (1 - \alpha)[\alpha Y_{t-1} + (1 - \alpha)Y_{t-2}] \\Y_t &= \alpha X_t + (1 - \alpha)[\alpha Y_{t-1} + Y_{t-2} - \alpha Y_{t-2}]\end{aligned}$$

Considerando che la moltiplicazione con α equivale a eseguire uno shift logico a destra, l'ultima equazione sviluppata è un buon punto di partenza per lo sviluppo del circuito, in quanto:

1. Le moltiplicazioni con α possono essere svolte in un solo ciclo di clock tramite un **barrel shifter**.
2. La prima serie di somme/sottrazioni incluse all'interno delle parentesi quadre può essere svolta attraverso un **carry save adder**.
3. La restante porzione dell'equazione può essere calcolata attraverso le operazioni che abbiamo già elencato.

Considerando anche che la sequenza di diversi componenti possa risultare "pesante" per il sistema generale, dividiamo le operazioni grazie all'implementazione di una pipeline tra il calcolo della prima somma e quello della seconda. Quindi, nel nostro caso, il circuito sarà diviso nel calcolo delle seguenti porzioni:

$$\begin{aligned}\text{Sum}_1 &= \alpha Y_{t-1} + Y_{t-2} - \alpha Y_{t-2} \\Y_t &= \alpha X_t + \text{Sum}_1 - \alpha \text{Sum}_1\end{aligned}$$

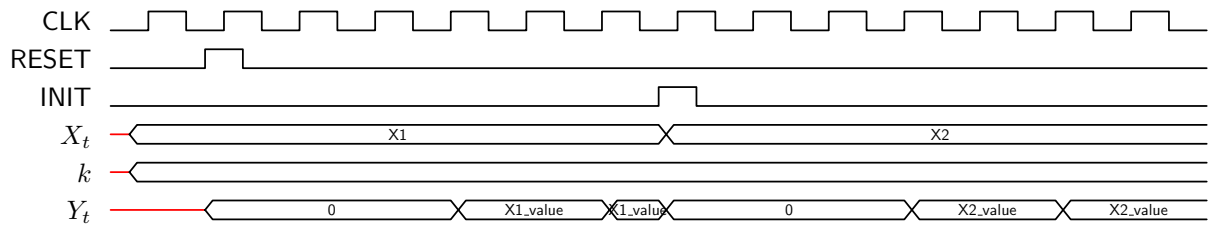
Infine, l'ultimo accorgimento che dobbiamo considerare è la presenza di una sola sottrazione all'interno delle due somme tramite **carry save adder**, la quale può essere semplicemente implementata all'interno dell'apposito modulo.

2.1 Interfaccia del sistema

Il nostro sistema avrà 5 segnali di ingresso e 1 solo segnale di uscita:

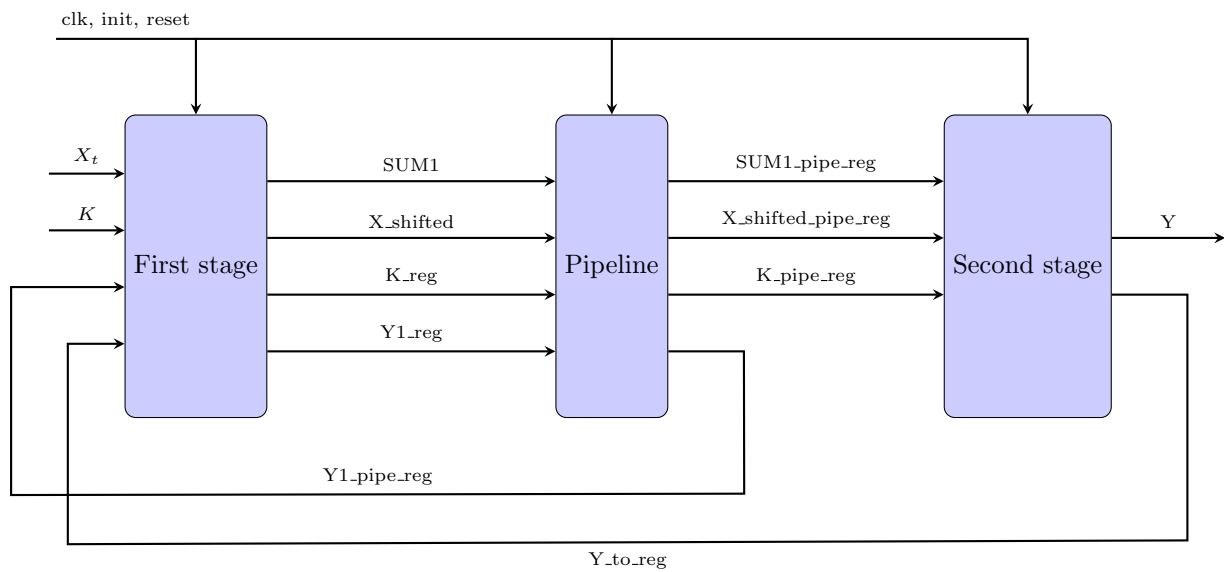
- Segnale X_t : segnale di ingresso rappresentato in formato binario su 32 bit, con 1 bit per il segno, 15 bit per la parte intera e 16 bit per la parte decimale.
- Segnale k : segnale di ingresso rappresentato in binario senza segno su 3 bit, sufficiente per rappresentare valori interi compresi tra 0 e 7, corrispondenti all'esponente utilizzato per il calcolo del coefficiente $\alpha = \frac{1}{2^k}$.
- Segnale clk : segnale di ingresso su 1 bit che rappresenta il clock del sistema, usato per la sincronizzazione delle operazioni interne.
- Segnale **reset** segnale di ingresso su 1 bit che funge da inizializzatore. Viene utilizzato solamente all'inizio del processo di calcolo.
- Segnale **init**: segnale di ingresso su 1 bit che funge da inizializzatore durante il processo di calcolo. Quando attivo, riporta lo stato del filtro a quello iniziale.
- Segnale Y_t : unico segnale di uscita, anch'esso rappresentato su 32 bit in virgola fissa, con 1 bit per il segno, 15 bit per la parte intera e 16 bit per la parte decimale. Contiene il valore del segnale in uscita dopo l'elaborazione tramite il filtro esponenziale.

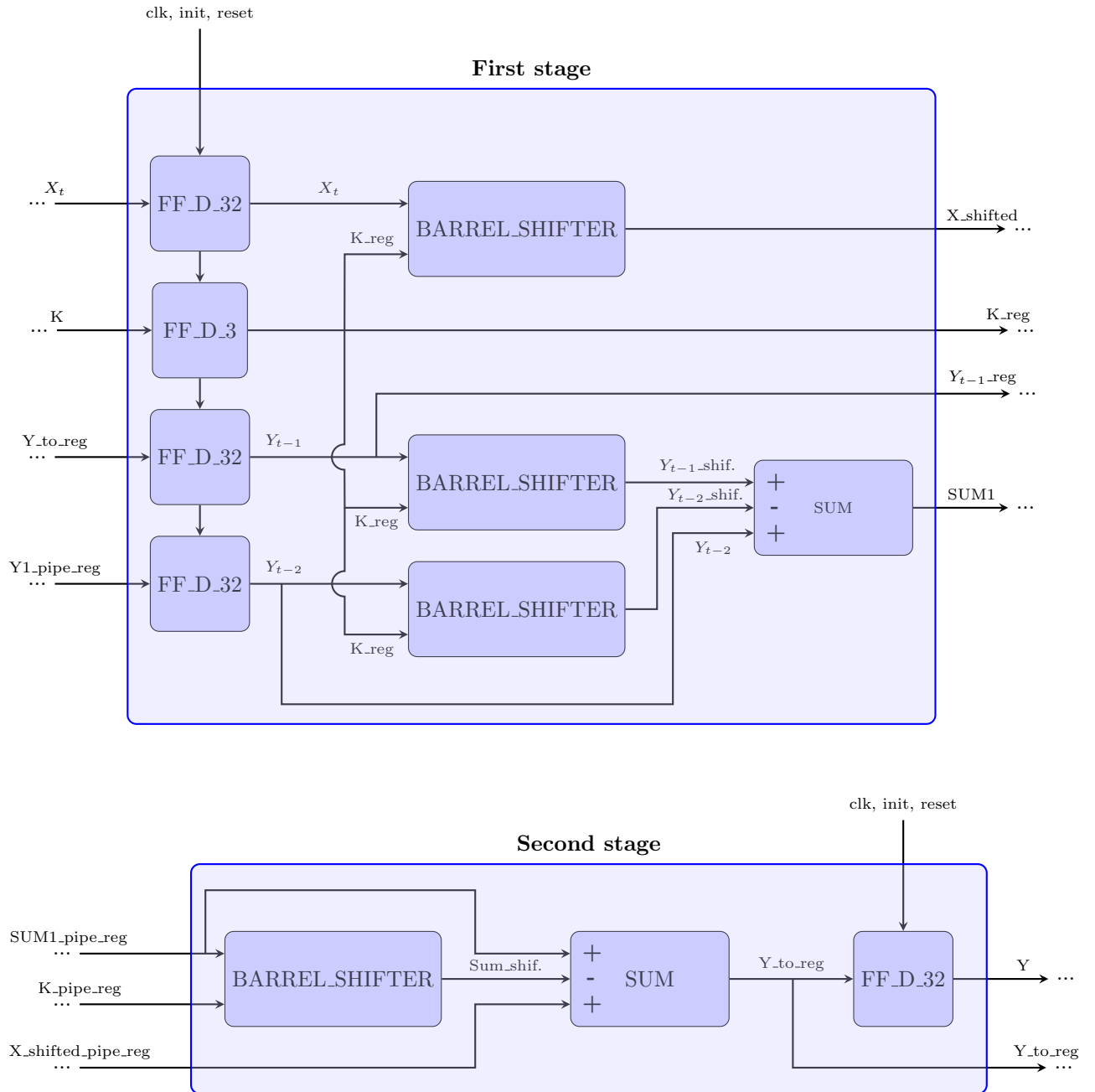
Per meglio rappresentare il legame temporale tra i vari segnali, analizziamo un piccolo diagramma temporale dei segnali:



- Finché **RESET** non viene posto a 1, il segnale Y_t non potrà mai essere definito, poiché gli stati passati (Y_{t-1} e Y_{t-2}) sono ancora indefiniti.
- Appena **RESET** viene posto a 1, tutti i registri vengono impostati a 0 in modo asincrono.
- Anche se i valori di X_t e di k erano già all'interno del nostro sistema, per poter produrre una prima uscita bisogna aspettare 3 cicli di clock dopo l'inizializzazione, poiché essi devono essere letti, elaborati e memorizzati prima di poter essere visualizzati.
- I valori calcolati successivamente al primo valore impiegano solamente 2 cicli di clock per essere visualizzati.
- Quando il segnale **INIT** viene posto ad 1 tutti i registri vengono di nuovo impostati a 0, per poi iniziare un nuovo sviluppo di risultati.

2.2 Architettura del sistema





Descrizione generale

Il sistema riceve in ingresso tre segnali principali: X , contenente il dato da filtrare, K , un parametro di controllo a 3 bit che regola il coefficiente di attenuazione del filtro, e i segnali di controllo RESET INIT e CLK. L'uscita del filtro è rappresentata da Y .

L'architettura è composta da più sottoblocchi funzionali, riutilizzabili e modulari, tra cui:

- **Registri a 32 e 3 bit:** impiegati per la sincronizzazione dei segnali X e K e per mantenere i valori intermedi del filtro.
- **Barrel shifter:** utilizzati per eseguire divisioni rapide per potenze di due, necessarie per il calcolo dei termini shiftati.
- **SUM:** adibito alla somma di 3 termini simil carry save, risparmiando tempo invece di eseguire somme successive.

Funzionamento del filtro

1. Gli input e gli stati passati vengono registrati all'interno dei registri.
2. Il dato X viene shiftato secondo il valore di K .
3. I due stati precedenti vengono anch'essi shiftati e combinate tramite somma.
4. I primi due risultati parziali, K e l'ultimo stato precedente (Y_{t-1}) vengono registrati all'interno dei registri della pipeline.
5. la somma parziale viene nuovamente shiftata e sommata con la versione shiftata di X .
6. Il valore finale è registrato in Y , rappresentando l'uscita.
7. In caso di cambiamento del segnale **INIT** al valore '1', tutti i registri vengono impostati asincronamente al valore "0".

2.3 Modulo BARREL SHIFTER

Questo modulo realizza lo shift aritmetico a destra di un ingresso a 32 bit, controllato da un segnale di selezione a 3 bit (S). Il valore massimo di shift consentito è di 7 posizioni. L'operazione viene svolta in un solo ciclo di clock grazie a una struttura gerarchica composta da tre livelli di shifter, ciascuno in grado di spostare l'ingresso di una quantità pari a una potenza di due (2^0 , 2^1 , 2^2).

L'idea è quella di utilizzare i singoli bit di S come abilitatori per i vari stadi, in modo che l'uscita finale corrisponda alla combinazione richiesta. Ad esempio:

- Se $S = "100"$, viene eseguito uno shift aritmetico di 4 posizioni.
- Se $S = "010"$, viene eseguito uno shift aritmetico di 2 posizioni.
- Se $S = "001"$, viene eseguito uno shift aritmetico di 1 posizione.
- Per tutti gli altri valori di S , viene eseguita una combinazione dei casi precedenti, applicati in cascata.

Da questa descrizione si deduce che i tre stadi realizzano shift rispettivamente di 1, 2 e 4 posizioni, e vengono controllati dai bit di S :

- Lo shift di 1 posizione è controllato da $S(0)$.
- Lo shift di 2 posizioni è controllato da $S(1)$.
- Lo shift di 4 posizioni è controllato da $S(2)$.

Ogni stadio è implementato tramite una concatenazione di segnali: quando il bit di controllo è attivo, l'uscita viene formata duplicando il bit più significativo (bit 31) per mantenere il segno, mentre i restanti bit vengono traslati. In caso contrario, l'uscita dello stadio coincide con l'ingresso.

Il funzionamento interno è quindi il seguente:

- Nel primo stadio (controllato da $S(0)$), l'ingresso viene traslato di 1 posizione e il bit 31 è replicato.
- Nel secondo stadio (controllato da $S(1)$), l'uscita dello stadio precedente viene traslata di 2 posizioni, replicando i bit più significativi.
- Nel terzo stadio (controllato da $S(2)$), l'uscita viene traslata di 4 posizioni, ancora con riempimento dei bit più significativi.

In questo modo è possibile ottenere qualsiasi shift da 0 a 7 posizioni combinando opportunamente i tre stadi.

2.4 Modulo SUM

Il modulo SUM implementa un carry-save adder a 32 bit. Esso consente di effettuare l'operazione $A - B + C$ in due stadi logici principali: un primo stadio CSA e un secondo stadio di somma ripple-carry. L'architettura si basa su due livelli principali:

1. **Stadio CSA:** utilizza 31 full-adder e un sommatore semplice per calcolare in parallelo le somme parziali tra i bit di A, B_IN ($B \text{ XOR } \text{CARRY_IN}$) e C, producendo due vettori: CSA_SUM e CSA_CARRY_OUT.
2. **Modifica segnale:** il segnale RCA_Y, che rappresente uno degli addendi dello stadio RCA, prende come primo bit '0' e per gli altri prende quelli di CSA_CARRY_OUT traslati di una posizione.
3. **Stadio RCA:** i due vettori prodotti dal CSA vengono poi sommati mediante una struttura ripple-carry, sfruttando 31 full-adder e un sommatore semplice alla fine, in quanto non necessitiamo del CARRY_OUT.

3 Verifica

Per la verifica del corretto funzionamento del filtro esponenziale, ho sviluppato due tipi di simulazione:

- **Behavioral simulation** : In questo tipo di simulazione viene verificato il corretto funzionamento del filtro tramite l'analisi degli output e la verifica della loro correttezza. Per questa verifica ci appoggiamo alla generazione dei risultati tramite un piccolo programma C, il quale scrive su un file di testo i risultati attesi che vengono presi da vivado e visualizzati accanto agli output effettivi del sistema durante la simulazione.
- **Post place & route simulation** : In questa simulazione, invece, si andrà a verificare il comportamento del filtro tenendo conto dei ritardi della rete combinatoria e della frequenza del ciclo di clock.

3.1 Test-bench

Per la behavioral simulation, viene istanziato il componente `FILTRO_ESPONENZIALE`, con segnali di ingresso e uscita rappresentati tramite vettori a 32 bit. Il clock è generato con un periodo di 2 ns (valore comodo per la visualizzazione dei risultati, dato che per valori di ingresso alti possono essere impiegate anche diverse interazioni per raggiungere la stabilità), mentre l'ingresso `RESET` viene inizializzato all'inizio della simulazione per impostare lo stato iniziale del filtro.

Un processo dedicato (`LEGGI_E_VERIFICA_Y`) legge dal file di testo i valori di uscita attesi e li confronta con il segnale `Y` prodotto dal filtro. Questo confronto permette di verificare la correttezza del comportamento del filtro nel dominio temporale.

Nella simulazione post place & route, `TB_FILTRO_ESPONENZIALE_PPR`, viene simulata un'esecuzione più vicina alle condizioni reali post-place-and-route, introducendo una fase di attesa iniziale (100 ns) per simulare la fine del GSR (Global Set/Reset). In questo scenario, il segnale di clock viene posto ad un periodo di 8.5 ns, valore ottenuto dall'analisi temporale statica (il critical path ha un tempo di setup pari a 7.125 ns) con un margine di circa il 20%.

3.2 Casi d'uso

I casi d'uso simulati coprono due principali scenari di funzionamento del filtro esponenziale:

- **Verifica funzionale con confronto su file:** nella behavioral simulation si verifica la risposta del sistema dato un ingresso costante e un valore arbitrario per K . I valori di uscita attesi sono precalcolati e memorizzati in un file esterno, che viene letto dinamicamente durante la simulazione per il confronto. Questo permette di testare la convergenza del filtro verso il valore di ingresso. Inoltre si può verificare la correttezza del filtro con valori di ingresso limite, infatti se il valore di X è minore di 2^{-16+k} l'uscita sarà sempre uguale a 0, questo poiché avendo una precisione limitata nel primo calcolo dell'uscita avremo:

$$Y_t = X_t \gg k$$

Dato che i valori Y_{t-1} e Y_{t-2} sono inizializzati a 0

Questo genererebbe $Y_t = 0$ se il valore di ingresso non è abbastanza grande e dato che i gli stati passati verrebbero aggiornati sempre con il valore 0, non otterremo nessuno sviluppo del risultato.

- **Simulazione PPR temporizzata:** nella simulazione post place & route si riproduce un contesto operativo in cui il filtro viene inizializzato con un valore di ingresso, lasciato evolvere per un certo intervallo di tempo, e poi sottoposto a un azzeramento dell'ingresso o ad una variazione di esso. Questo scenario consente di osservare il comportamento dinamico del filtro a fronte di variazioni rapide dell'input in condizioni di temporizzazione realistiche. Si può notare, infatti, che il segnale di uscita non varia istantaneamente con il ciclo di clock ma ci mette un tempo pari a $t \in [3.302ns; 3.314ns]$ prima di stabilizzarsi completamente.